Six Dimensions of Benchmarking Time-Series Databases

1st Sandro Melissano
Institute for Process Data and Electronics
Karlsruhe Institute of Technology
Karlsruhe, Germany
sandromelissano@gmail.com

2nd Jalal Mostafa
Institute for Process Data and Electronics
Karlsruhe Institute of Technology
Karlsruhe, Germany
jalal.mostafa@kit.edu

3rd Suren Chilingaryan
Institute for Process Data and Electronics
Karlsruhe Institute of Technology
Karlsruhe, Germany
Suren.Chilingaryan@kit.edu

4th Nicolas Tan Jerome
Institute for Process Data and Electronics
Karlsruhe Institute of Technology
Karlsruhe, Germany
nicholas.tanjerome@kit.edu

***Abstract*—Time series databases have each their specialization and inner concepts, which run sooner or later into individually different performance bottlenecks. When designing data backends with requirements that increase with time, system architects want to understand which storage engine is more efficient for their specific data pipelines and shows less risk to run into future bottlenecks. We present the benchmark SciTS-v2, which compares storage engines' performance and especially efficiency profiles under six distinct workload characteristics, like equidistant, uni- and multivariate, and mixed workloads — all present in the use case of scientific experiment's monitoring systems. Our evaluation points out optimal workload configurations for the tested systems.**

***Index Terms*—time series, database engines, benchmark, vector databases, time series database engines, Monitoring Systems, IOT**

I. Introduction

More and more scientific infrastructures use monitoring, analysis and AI-based automatization — the efficient management of time series plays an important role in these and many other monitoring applications. Especially in the scientific domain, where experiments consist of large amounts of sensors, data need to be stored as optimized as possible. Storing and managing higher amounts and rates of time series data, while saving storage and maintenance costs, is a crucial mission not only for scientific but for data backends, where high-availability, scalability, and comprehensive online-analysis capabilities are requirements. As the amount and frequencies of time series increase, data increase both in bandwidth and in storage consumption. While higher bandwidths expose the storage engine's bottlenecks, higher storage consumption is one of the main costs factors. To provide continuous online analysis, systems need to perform well in parallel read and write workloads. Compound systems, which deliver these requirements via replications, require more computing and maintenance infrastructure than storage engines, which are optimized to deliver competitive online-analysis capabilities in one single instance – with maintenance costs being one of the main cost factors. Emerging applications, such as scientific analysis and data stream automatization using machine learning techniques, need high raw data output bandwidth, low response times, and vector data formats. This research targets integration, performance, and efficiency issues by analyzing different time series specialized storage engine architectures. We test systems which are currently in use by related experiments, as well as an emerging storage approach to assess the suitability of specific implementations of storage engines for the use case of scientific experiments.

a) Motivation and Contribution:: Our team's research follows the vision of the experiment's data being seamlessly integrated from data acquisition to analysis — thus bringing data from the sensor directly to storage with minimum hardware requirements. Despite previous studies in the field of time series database benchmarking, summarized in Table I, there is still:

(1) a need of a qualitative analysis on all kinds of system resource consumption to understand how the various architectures manage the different workloads. Previous studies, as shown in Section III, concentrate on performance comparison, with limited analysis of system metrics, present no insights into effects of write amplification or computational efficiency as trade-offs for a given performance. These assessments are important when building solutions with a long lifetime (up to decades). **(Contribution)** Our quantitative and qualitative analysis of system resource consumption and performance metrics of different architectures of time series specialized storage engines, selected as shown in Section II-A, evaluates how various engines run into bottlenecks with increasing requirements. **(2)** We want to measure the precise impact of storing various amounts of values under a single key on different storage engines. Databases use this strategy to improve performance, but this often comes with a more rigid schema, causing problems in continuously changing sensor en-

vironments. The question of how many values stored together are most attractive as a trade-off has not been targeted in recent time-series benchmarks. **(Contribution)** We benchmark multivariate workloads with a parameter, we call 'value-cardinality'. We fill tables with four different amounts of values stored in each data point and compare performance and consumption profiles. Based on our use case and setup, we point out an optimal configuration for this workload parameter for each tested system. **(3)** The aim of reducing storage and write amplification by eliminating time redundancy in equidistant time series appears as a very promising approach for use cases with equidistant time series. No such storage engines have been benchmarked yet. **(Contribution)** We benchmark an emerging equidistant time series specialized array-based storage engine: DataLayerTS. We investigate how every system manages both equidistant (regular) and non-equidistant (irregular) time series workloads.

(5) To conduct our research, we need a tool, which unifies an extensible, comprehensive, and computationally efficient software with investigational features, that compare time series storage engines under both mixed and non-mixed ('dedicated') workloads, single-value vs. (various) multi-value (multivariate) data point workloads and equidistantly-vs.-irregularly distributed timestamps. In addition, a joint analysis of detailed system and performance metrics is needed to easily merge both measurement types. As shown in Section III, there is no available benchmarking tool that features all requirements. **(Contribution)** After comparing known tools (Section I, we developed the following investigational features based on the SciTS [1] benchmarking tool, published as SciTS-v2. It permits all required investigations. Furthermore, we analyze batch size, amount of parallel clients, and length of time series per batch. These six workload characteristics gave the name to this paper, as they represent 'six dimensions of benchmarking time series databases'. With this tool, we conducted a benchmark to find the best workload configuration for all tested engines and assess the most suitable engine for our scenario.

II. Background

Scientific experiments emit large amounts of time series, containing: (1) Configuration parameters, which are used to reconstruct the causes of any observed event; (2) Environmental information, necessary to determine external factors which contributed to any observed event; (3) Operational values, which contribute to operations, maintenance, or failure detection. Generally, these tasks are the main focus of monitoring systems of scientific experiments.

Scientific experiments, e.g., in the realm of high-energy physics research, consist of complex machines, which produce time series data: Configuration parameters, which continuously change during usage, need to be stored to be able to reconstruct the causes of what led to any observed event. On the other hand,they store detailed information on environmental information, necessary to determine external factors. Furthermore, internal process and system data contribute to operation, maintenance and failure detection.

In our scenario, inspired by the KATRIN monitoring system, these experiments can use thousands of sensors, producing 1 to 10 data points per second, active throughout the whole year.

Observations: Once the sensor data flow of our specific use case was analyzed, the KATRIN [2] [3] experiment, the following observations were made:(1) Many sensors emit data regularly, based on a specific time resolution. Some sensor boards generate values as an array, where the index positions are defined by time. Currently, an intermediate layer translates the Y-lenght arrays to Y independent rows for a relational table. (2) Often several sensors are strongly related and tied to each other: To understand a specific situation, all of these related sensors have to be queried – e.g., temperatures in a region. (3) The data is mostly queried as aggregations. Raw data is needed only in special situations; (4) During any ingestion workload, the system needs to read out both stored ranges of raw data and aggregations. Apart, emerging applications need time vectors of raw sensor data to train neural networks for prediction tasks.

a) Equidistant Time Series: Based on the first observation, we want to understand how storage engines manage specifically equidistant or, on the contrary, randomly-distant time series. This means that equidistant (also called 'regular') time series always have the same time distance between timestamps – e.g., 1 second. Irregular time series have a random, always different interval. If a series is in theory regular, but might have single non-regular signals, this could be still a regular time series just if the following signals are not affected in their expected, regular timestamps, and the latency of the exception is not of interest (precision is not needed on that level). In this study, we configured the irregular time series data generator to add a random offset to each timestamp in a way that results in intervals of around 1 second on average. This ensures that the distribution of measurements in any time window is comparable to those of regular time series. We aim to understand whether an architecture is particularly performative with regular time series.

b) Sensor Index Reduction and Value-cardinality: Especially in row-based systems, discussed further in Section II-A, but possibly also in other architectures, storing a sensor group as multivariate time series (multiple values under one key) has shown to be more performative than storing these sensors individually. Having fewer sensor index keys the index gets smaller, the amount of pages fewer and reads and writes faster (as the candidate key specific to time series databases is [timestamp, sensor ID]). This makes particularly sense if these measurements are also queried together, e.g., in the case of an area of temperature sensors. However, in experiments with a long lifetime, updating, changing, or reconfiguring these sensors has shown to be tedious, due to the fixed schema. If and how to implement this strategy in future systems is a design decision which is supported by our benchmark data. For each architecture in our setup, we point out which are the performance and efficiency gains (or losses) with increasing

group size.

To differentiate these sizes, we introduce the term 'value-cardinality', which indicates the precise amount of sensor values in a group, saved as an array in a single data point or as columns under a single ID key. Conventionally, cardinality means how many distinct primary keys are present in a table. Value-cardinality means how many distinct values are present under a distinct identifier key in a table and must be defined for each key (or for all keys in a table, like we did in this benchmark). While this could be covered by the term multivariate time series, we note that our scenario's data is not multivariate data; given workloads just treats the data like multivariate data to optimize storage engine's efficiency or performance. We choose 6,12, and 24 values, as amounts of values in this range are stored together in current data base schemas in our use case.

A. Key Differences across Storage Engine's Architectures

Prominent experiments and monitoring systems currently rely on various database systems, which have been in use for many years, listed in the following paragraph. From this collection, we selected a system for each architecture category: Row, key-value, column. Our criteria to choose these systems are a large user base, long term support, and good documentation, as our use case requires sustainability. Besides, we include an emerging vector-based architecture type, which is appealing due to the promised strong reduction of time index overhead. This will show, whether this vector architecture type is promising and a valuable approach for future database engine developments, which one day could become a sustainable solution for scenarios like our use case. E.g., vector architectures are particularly interesting for machine learning-based analysis and applications, as large vectors can be loaded directly into integrated high performance computing accelerated analysis pipelines or model training.

Oracle, MySQL, MSSQL (rowstore), PostgreSQL, and TimescaleDB are well known **row-based** database system. MySQL and MSSQL are currently used by the KATRIN experiment. MySQL and Postgres have been benchmarked by the first SciTS benchmark [1], in which a critical decrease in ingestion speed with ongoing ingestion has been observed. Like Oracle, all three are general-purpose, not time series-specific database systems. Therefore, we choose TimescaleDB. **TimescaleDB** (TS) is a Postgres extension, based on standard row-based Postgres 8 kB-"slotted-page"-blocks. In detail, Postgres provides a list of pointers in the beginning of a file (block), and the entries (the values) are written from the end of the file, so that the unallocated space is in the middle. Like in any row-based storage engine, values of the same key(s) are stored next to each other – independently of the type (Therefore, a schema must be defined). The novelty is that Timescale sorts the files by time and splits them, with these child-tables being organized by a Hypertable, making it faster to read and write, if time is your main key. To update this B-Tree, the related 8 kB pages have to be rewritten. Timescale has another particularity: As soon as a configurable period is passed, data is not considered "hot" or somehow involved in "live" operations, and subsequently compressed. This compression changes the data format to a column-based format. As the Postgres server needs to read the data again in its own format, once accessed, the columns get re-converted to the row-based "slotted page", which takes additional resources. This is ideated for rarely accessed tiers. In our use case, the system deals with continuously high data loads and needs to compress. Additionally, the archived ("cold") data is intensively accessed by the users. Therefore, in our benchmarking experiment, we decided to column-compress all incoming data instantly on arrival. We measured storage consumption of 1-Day data of all tables for a column-compressing DB and a non-column-compressed DB, shown in II.

OpenTSDB(on top of HBase), HBase (used by Hefei Experiment, China ADS Linac, Japanese Proton Accelerator [4] [5]), GridDB, InfluxDB (used by ISIS and the Los Alamos [6]), MongoDB (used by Beijing Electron Positron Collider II [7]), CrateDB and AsterixDB are known **key-value**-based database systems. Influx outperforms OpenTSDB-HBase in performance and disk consumption (Hao et al. [8]) and outperforms also GridDB, CrateDB and AsterixDB (Gupta et al. [9]).[1] At the time of experiment (winter 2024/25), Influx V3 OSS was not yet released and we choose Influx V2.

InfluxDB (IF), a key-value store, writes all keys and their values in a sorted list, which is sorted by time and primary key, so no schema is necessary. This list is generated first in memory and then flushed when it reaches a certain size. With incoming updates, a new list is created and indexed, and the old one is merged with other older entries. This procedure is analog to an LSM-Tree [10]. To retrieve a given data point, multiple files might be accessed until the most recent entry is found. Influx is schemaless, which is a very interesting characteristic for long-lasting systems that change requirements (e.g., sensor distributions) over time.

KairosDB (on top of Cassandra), Cassandra (used at Lawrence Berkeley National Lab. [11]), QuestDB, ClickHouse (used by CERNS's LHCb in 2012), VictoriaMetrics (used in CERN's CMS), MonetDB, MSSQL (columstore) and Druid are known **column-based** database systems. KairosDB (by Lie et al. [12]), QuestDB and Druid (by Khalifati et al. [13]) have been evaluated as less performative than ClickHouse. Apart, Khelifati et al. [13] assess MonetDB as the least performative system. ClickHouse and VictoriaMetrics both use the ClickHouse storage engine concept, with VictoriaMetrics focusing on integrating with the Prometheus software stack. However, we selected ClickHouse as the more established variant. [2]

ClickHouse (CH), a column-based store, stores values together (in the same block), which have the same data type — means, are in the same column. To index them, it sorts the time and the key with its values, and splits them in arrays

[1]In addition, document stores like MongoDB are not optimized for monitoring scenarios like our use case.

[2]In addition, wide-column stores with column families, like native Cassandra, are not optimized for monitoring scenarios like our use case.

of fixed size, "Granules": Data blocks of 10^{12} bits, which are claimed to fit well in CPU caches [14]. The index saves just the start time and key of both the time and key-granule and maps them to their relative values-array.

KDB+ (stored on DISK), Milvius, Pinecone, DataLayerTS, TileDB and SciDB are **vector-based** database systems. Milvius and Pinecone fokus on AI/Tensor applications, KDB+ originates in an in-memory database. TileDB (which outperforms SciDB [15]) and DataLayerTS are both interesting for this research. We chose DataLayerTS, because it is more focused on time series data, and used in comparable data environments, like solar parks.

DataLayerTS (DLTS or just DL, in plots with no space), a vector-store specialized on equidistant time series, brings this idea to the next level with eliminating the time-data by introducing a pattern. As many applications just need an approximate time information, as they store regularly taken measurements, DataLayerTS saves just the start time, the length, and the time resolution of a time series. The values are written in a per-sensor WAL, until the checkpoint process adds the recent data to the history array (one long array per key) and compresses it. That means, the system stores long, brotli-compressed arrays (one for each sensor ID) of pure double float values in directories, with meta-data attached to it. To extract a specific time-value pair, the whole array needs to be unpacked, the meta-data opened, and the requested index position calculated. This calculation is done with the array's start-time and the array's time resolution saved in the meta-data file. This explains why no irregular time series data is supported. This system works primarily with batches of consecutive time windows, as the new WAL-data can be appended to the existing array. Historical data require a heavy array rewrite and is therefore to be inserted via the special irregular-data API and saved as non-vector-based files.

III. Related Works

Benchmarking database systems specifically for given application's requirements is crucial to select a suitable database system. In Table I we compare named benchmarking tools for their support of the required features. Several research groups and developers created time-series benchmarks.TPCx-IoT and YCSB-TS are based on the YCSB suite. While TPCx-IoT [16] features just basic queries, YCSB-TS is more extended and tests 11 systems. Rui Lui [12] introduced the IoTDB benchmark. It compares Influx, Timescale, OpenTSDB and KairosDB [17]. SmartBench [9] evaluates seven systems [3]. Hao et al. [8] proposes the ts-benchmark, testing InfluxDB, TimescaleDB, Druid and OpenTSDB. TSM-Bench [13] by Khelifati et al. tests seven systems (ClickHouse eXtremeDB InfluxDB MonetDB QuestDB TimescaleDB and PostgreSQL). Moreover, commercial database systems' vendors have conducted benchmarks for their and competing products: InfluxDB-comparison [18] and it's fork Time Series Benchmark Suite (TSBS) [19] compare based on storage size, loading performance and aggregation runtime. ClickBench [20], proposed by ClickHouse, evaluates DBMS via simple queries, based on simple scripts.

The following benchmarks have been designed and conducted for scientific instrumentation and monitoring systems, developed from various experiment's collaborations. Vasile et al., part of CERN's ATLAS experiment, tested Influx and ClickHouse. Chen et al. [21] conducted a benchmark for the HALF-synchrotron. However, both do not present a dedicated benchmarking tool. SciTS (v1), introduced by Jalal et al., [1] is a TSDB benchmark tool, implementing various queries interesting for scientific instrumentation like sampling and aggregations and have been designed in collaboration with the KATRIN physicists.

In Table I, we show a comparison between these tools, which points out, why SciTS-v2 is the most suitable tool to conduct this research.

BM-Tool	YCSBTS	SmartB.	IoTDB	ts-bm.	SciTS	TSM-B.	TSBS	SciTS-2
Reg. / Irreg. WL			✓					✓
Value Cardinality								✓
Mixed Workloads	(*) [4]	✓				✓		✓
Nr. of Clients / Batch Size	✓ /		✓ / ✓	/ ✓	✓ / ✓	/ ✓	✓ / ✓	✓ / ✓
Distinct Time Windows			( )[5]	✓	✓	( )[6]		✓
Time Resolution	✓		✓				✓	✓
Sensor Cardinality	✓[7]	✓	✓	✓	✓		✓	✓
Modular Data Generator		(*)[8]	✓	✓	✓	✓	✓	✓
Optim. (API) Writes[9]	✓		✓		✓	✓		✓
Unified Sys.& Perf. Metrics[10]					✓			✓
FS & Disk Metrics		✓	✓		✓	✓		✓
Extended CPU & RAM Metrics					✓			✓
Network Metrics					✓			✓
Simple / Complex Aggreg.	✓ /	✓ /	✓ /		✓ / ✓	✓ / ✓	✓ / ✓	✓ / ✓
Filter Queries		✓	✓	✓	✓	✓	✓	✓
Supported DBMS	2	2	2	2	3	3	3	4

✓ = feature exists, (*) = feature exists to some (small or unsatisfactory) extent – see comment,
( ) = feature missing or not suitable – see comment

TABLE I: Benchmark-Tool's Feature Comparison Table

To assess which storage engine is most suitable for our use case, this paper proposes the SciTS-v2 benchmark tool. It is an overhaul of the SciTS benchmark tool, presented by Jalal et al. [1].

IV. The Benchmarking Architecture

All benchmark tools in Section III have been evaluated with the set of requirements. A full comparison is shown in Table I. SciTS-v1 provides the following key advantages: (1) Patchwork-Ingestion: In this mode, although batches are

[3] PostgreSQL, AsterixDB, MongoDB, GridDB, CrateDB, SparkSQL, InfluxDB

[4] operation proportions can be set, which simulate to a small extent mixed WLs

[5] It follows the concept of inserting actual timestamps, based on data-import-completed time.

[6] either load a dataset or take the GAN-output, which needs to be trained

[7] via tagcount

[8] data is not generated, but read from directory

[9] it is acknowledged that not all DBMS's might offer APIs to all Plattforms.

[10] means, have a common key or are stored together. Necessary to automatically attach the measurements to the workloads configuration.

made internally from consecutive timestamps (like in most of the benchmarks), these batches each carry a different randomized time window. In contrast, "consecutive-batches" are considered the default way of managing time series. (means, batches, where the values represent consecutive time windows) (2) CPU-optimized Architecture and Runtime Environment: Designed as both multi-threaded and asynchronous task handling software, SciTS leads to a better usage of available computing resources. (3) For the KATRIN experiment, no reliable data predictions for generators or representative small data sets are available, so we prefer high-entropy random data generation. Furthermore, the system needs to be able to configure various parallel clients for ingestion and query. The queries provided by SciTS are implemented as a result of a study in collaboration with the physicists of the KATRIN Collaboration [1]. (3) Fully managed metrics collection: No other tool has a seamlessly integrated metrics collection that integrates various types of metrics. Other tools limit themselves to performance tests and collecting server response latencies. System metrics are logged separately with arbitrary methods and tools and reconnected to the benchmark run's parameters manually. SciTS incorporates Glances [22] via Restful protocol and saves detailed system metrics and performance metrics under a unified ID, together with the current workload parameter configuration. Prior to SciTS-v2, just TSM-Bench benchmarked mixed/online workloads (one query type along with various insertion bandwidths under the same workload configuration). IoTDB-benchmark did a storage consumption comparison between regular and irregular timestamps. Therefore, these benchmark tool provide parts of the interesting features, but have not shown to be easily extendable and do not meet other requirements about metrics collection and workload configurability. SciTS-v2 introduces: (1) An equidistant/non-equidistant time series data generator, with both row and vector-based time series modes. (2) It generates data points with one or multiple values. (3) The workload manager accesses the specified tables across regularity and value-cardinality parameters or initially creates and configures them automatically. (4) SciTS-v2 supports uni- and multivariate raw data queries[11]. (5) The workload manager can launch any kind of query and ingestion workload concurrently, simulating any specified I/O requirement. The relation input-vs-output bandwidth usage as well as a selection of concurrent queries can be specified. (6) The benchmark unifies various metrics under a common iteration ID. (7) In this benchmark, every iteration continues to write starting right after the last timestamp of the last batch of the previous iteration, to not provoke overwrites, using a log file.

V. Experimental Setup

In our scenario, the data back-end is hosted in a scientific computing and data center facility, which provides access to large storage resources. This setup permits maintenance of mission-critical storage to be outsourced, and high performance computing infrastructure can be integrated on demand for specific analysis tasks. Therefore, we chose our experiment hardware to meet data center standards. Both client and server are distinct, dedicated server-class machines exclusively used by this experiment. The following hardware-benchmarks help to understand how the differences in the storage engine's performance measurements and the detected bottlenecks are correlated with the used computing resources. **The server** uses an Intel(R) Xeon(R) Platinum 8370C CPU @ 2.80GHz with 8 logical cores[12] (2 threads per core, 4 cores on 1 socket, 4x L1d cache: 192 KiB, 4x L1i cache: 128 KiB, 4x L2 cache: 5 MB, 1x L3 cache: 48 MB - under 1 NUMA node. Sysbench found with 1 thread 1145 (Avg: 0.87ms latency) and with 8 threads 4766.58 (1.67 ms latency) prime numbers per second). The server has 32 GB DDR4 ECC (1GB Batch Size: 1 thread memory performance: 10 GB/sec and 99 ms avg. latency, 8 threads: 49 GB/sec and 157ms latency avg.) and multiple 1 TB NVMe SSDs with XFS (8 threads writing: (ca.1700x8) 14k IOPS (4K BS) with 77k usec submission latency – 1.2 GB/s bandwidth(1M BS) with 1.2k IOPS. 8 Threads Reading: (ca. 7700x8) 61K IOPS(4K BS) with 5k usec submission latency – 1.2 GB/s bandwidth(1M BS, with 1.2k IOPS).).
The client uses an Intel(R) Xeon(R) Platinum 8370C CPU @ 2.80GHz with 64 logical cores (2 threads per core, 16 physical cores per socket, 2 Sockets, L1d cache: 1.5 MB (32 instances), L1i cache: 1 MB (32 instances), L2 cache: 40 MB (32 instances) L3 cache: 96 MB (2 instances), NUMA node(s): 2 - 38183 prime numbers per second (64 threads)), 512 GB DDR4 ECC. [13]. DataLayerTS supports irregular data just for exceptional use cases, e.g. backfilling, with very low performance (few MB/S). The same is true for patchwork ingestion, for which the seperate irregular-data-API has to be used. Therefore, these measurements are included only in Figure 2(c). As DTLS is primarily tested for exploratory reasons regarding equidistant monitoring data, we decided not to fully support irregular and patchwork workloads in this benchmark.

All measurements for ingestion and query workload performance, regarding Timescale, are applied on a "cold" / columnar-transformed, instantly compressed storage — as for our use case, requests are often done on archived data. The schemas and systems configuration can be seen in the repositories schema and scripts folder, in the deployment scripts. However, this benchmark worked mostly on standard configurations with Timescale's Postgres being tuned (timescaledb-tune) and PgBouncer as connection pooler.

VI. Results

All latency values are in microseconds. "Dedicated" here means (in all plots) "non-mixed/offline" workloads.

[11]Along with the priorly existing Out-of-Range query, aggregation on time (mean and standard deviation) queries, down-sampling, and the comparing two down-sampled series query.

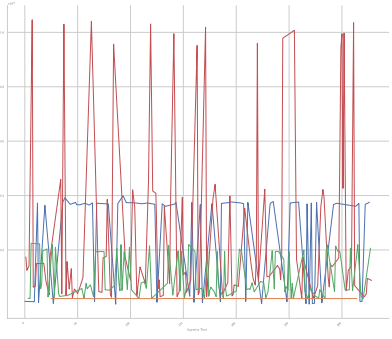
[12]the node is identical to the client node, just with only 4 physical cores out of 64 activated.

[13]These numbers are provided by Sysbench and Fio with run configurations and full output available in the repository

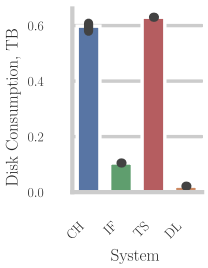
All values are in gigabytes.

System	TimeScale Live	TimeScale Cold	InFlux	Click-House	Data-LayerTS
Total Disk Usage [14]	146	59	55	42	21 !(x2)[15]

TABLE II: Storage consumption, data-directory only.

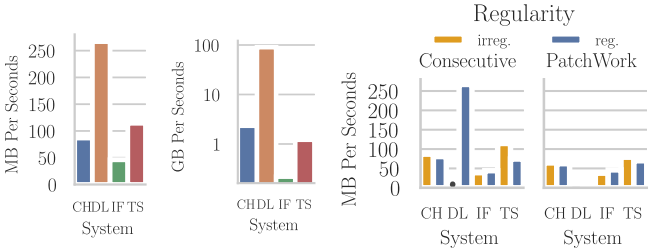


(a) During 1-Day data ingestion.



(b) After 1-Day data ingestion and querying.

Fig. 1: Total file system usage.



(a) Ingestion (b) Query (c) Patch Work Mode vs. Consecutive Time Windows

Fig. 2: Top performance comparison, each system listed with its most performative workload configuration. Offline.

A. *Storage Efficiency*

Reducing storage consumption is crucial to saving storage medium costs, especially in big data environments. The first measurement we present compares the consumption of disk space of one day of data, across various value-cardinalities.

Table II shows the storage consumption for all 8 tables, for the smallest (1 val.), and the largest (24 val.) table, all filled with the "populate" function of the benchmark tool. Each table contains 1 day of data with 1 second resolution from 1000 sensors - therefore, 86400 (seconds per day) x 1000 data points. We test various value-cardinalities using various tables: 1 value per data point, 6-,12- and 24 values per data point are inserted in their respective tables for each of the 1000 sensors and seconds of the day. Multiple values per data point mean that, by this factor, more data take up space in a table. On top of that, we test equidistant and irregular time series. Therefore, we have for every value-cardinality one table filled with equidistant (regular) and a separate one with irregular time series. That makes in total 8 tables. In theory, each day has 86400 seconds and each table 1000 sensors for which 1,6,12,24 double float value(s)(8 B) is/are stored: 691.2 MB of values per value-cardinality. (without timestamp or key) When adding the integer sensor ID, 345.6 MB, and the timestamp in nanosecond precision, 691.2 MB, the data occupies theoretical 1.7 GB for one saved value per data point, 5.2 GB for 6, 9.3 MB for 12 and 17.6 GB for 24 values saved in a data point. The observations have been measured after writing-to-disk ("populate") process, so no compaction has been done yet. For Timescale-Cold, it has been re-measured after the column-transformation / compression has manually been executed.

[14]for all Tables: 2x (both regular and irregular TS-types, apart DLTS (just regular)) 1-, 6-, 12- and 24- values per data point tables]

[15]DLTS just tested with regular time series, meanwhile all other DBs saved all data as regular and as irregular TS tables. So this value respects half of the data of the other DBs.

We observe that Timescale has the highest consumption in both live [16] and cold[17] data modes, although the cold storage becomes almost as small as its two closest competitors. These concepts are explained in Section II-A. For Live mode, we find with 146 GB a remarkably high number: This could be because PostgreSQL allocates 8 KB pages, even if these are just sparsely filled. In addition, it holds multiple B-Tree indices on disk for each table. Influx instead manages to store the same data in half of the space, due to the better data compression they can enable on the TSM, a sorted list with a bigger file format. In addition, indices are smaller, as they use the file structure to find keys. However, ClickHouse, with its native column-based, sparse-index architecture, manages to outperform Influx by 31% and Timescale by 40%.

Figure 1(a) shows the file system usage during the "popilate" ingestion. This means inserting 1 Day of data for 1000 sensors, all value-cardinality-tables and regular/irregular tables. 100 clients, 600 batch size). We note that TimeScale, but also ClickHouse use large amomunts or the drive for their merging operations. DLTS instead, but also Influx do not show this trend and need less space.

In Figure 1(b) we see the file disk usage after the populate-ingestion and after the Query benchmark. This means that all indices have been built. We see that ClickHouse and TimeScale use most space for this. DataLayerTS instead uses only the space it needs for the data, plus some megabytes of index overhead. This is due to DLTS having a very minimal index architecture, which is just suitable for finding sensor and monitoring data. As a consequence, any overhead is dropped, which comes to building multiple B-Trees, allocating fixed-size pages (even if they are just sparsely filled), or holding multiple tiers of MergeTrees. Therefore, it is not necessary to allocate huge amounts of space for these indices. As data is directly stored as very few, big arrays, compression can be as efficient as possible.

B. Top Ingestion and Query Performances across all configurations

The second analysis we present in Figure 2 shows the top performance of the distinct systems (offline), each one under its individually best workload configuration. The parameters are: Batch size (just ingestion), client numbers, and amount of values per data point. As we can see, DataLayerTS provides outstanding speeds for both ingestion and range queries. After Timescale and ClickHouse, Influx appears in this analysis as the slowest system. However, this research analyzed many workload parameters, of which just few are applied in a given use case. This means that the speeds shown are not given in any case. On the contrary, the more specialized a system is (like DataLayerTS), the better it performs in a narrow parameter set, while it is not performative outside this set. In this section, we compare the speeds of each system's best configuration parameter set, which are each made for very different use cases.

To better understand under which circumstances these top speeds appear and to understand, which performance is to expect under different configurations and to assess the various trade-offs and strengths, as well as the efficiency of the four systems, we will look at each workload parameter in detail.

In the following analysis, for DataLayerTS, just equidistant time series data workloads are included. The maximum performance for irregular data was around 2 MB/s and is supported only for exceptional cases.

C. Ingestion Performance

1) Ingestion Performance Under Regular And Irregular Time Series: All ingestion metrics displayed describe the ingestion of batches with consecutive time windows – with the exception of .

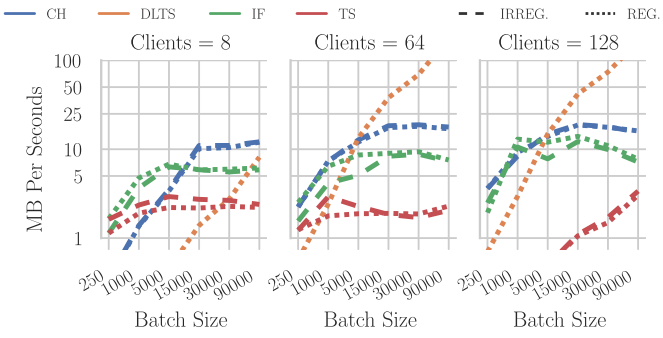


Fig. 3: Performance Comparison between equidistant (regular) and non-equidistant (irregular) time series ingestion across various batch size and parallel clients configurations, for one value-per-data point workloads

In Figure 3, we observe that Timescale (compressed) is less performative than ClickHouse and Influx, DataLayerTS is most performative on high client numbers and large batches, but has the lowest performance measured for low batch sizes and client numbers. Timescale, if it is under many clients, performs poorly, especially on small batches.

DLTS is less performative for low client numbers or batch sizes. This is due to its tuning, which is optimized for high client numbers and batch sizes: The system gets a higher amount of consecutive timestamps from the Memory to the Write Ahead Log (called "head" file) before doing the checkpoint, which means here: Merging more data to the compressed "history" file vector. This saves write-amplification (I/O-Usage), as for every checkpoint, the array needs to be fully rewritten, which is the weak point of DLTS. Rising the checkpointer frequency will improve speed for small batch size configurations, but results in a tinier I/O-bottleneck and lowers DLTS's speed for big batches. As we see in Figure 5(d), write amplification is an important weakness in DLTS. Generally, we observe that most databases reach their higher range of performances around the batch size of 15000 - only DLTS scales further, if using higher batch sizes. This parameter value will be important in Section VI-D1.

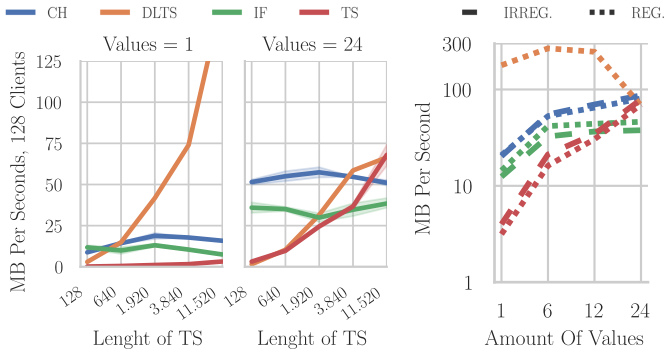
2) Ingestion Performance Under Consecutive Ingestion: In Figure 2(c) we observe the overhead of various systems that comes with PatchWork ingestion, means: handling subsequent batches which are not of consecutive time windows (the batches internally continue to consist out of consecutive timestamp). Vice versa, we see the performance benefits of architectures, which are optimized for seamlessly consecutive time windows batch ingestions. In this figure, we see the top speeds across averages of every ingestion parameter combination.

We observe that DataLayerTS performs by magnitudes better on consecutive batches (DLTS PatchWork ingestion became incredibly slow with increasing database size: Instead, for PatchWork-like use cases, the irregular data ingestion API has to be used). ClickHouse shows a slight improvement, while Timescale's irregular TS ingestion speed is notably increased under consecutive ingestion.

In theory, the usage of a WAL-MemTable minimizes the effects of various kinds of batch ingestion, as all data gets first sorted in memory, before written to disk. DLTS has a per-series-WAL, which merges with a history file. Therefore, ingesting non-consecutive data means here that whole vectors need to be unpacked and rewritten, instead of being able to merge data in an appending-focused routine, like it would be the case for consecutive batches of data. That explains DLTS's crucial difference. Instead, the other systems can, with little overhead, insert a patchwork-ingested batch just by rearranging some index keys, letting them point to the new, arbitrarily placed files.

3) Ingestion Performance Under Various Data Cardinalities: In Figure 4(b) we see the performance of data ingestion across all value-cardinality configurations, as an average of the other parameters (batch size and client numbers).

[16]the original mode, before converting to columnar format
[17]after transform/compression



(a) TS length per batch

(b) Values per data point

Fig. 4: Special ingestion workload parameter comparison.

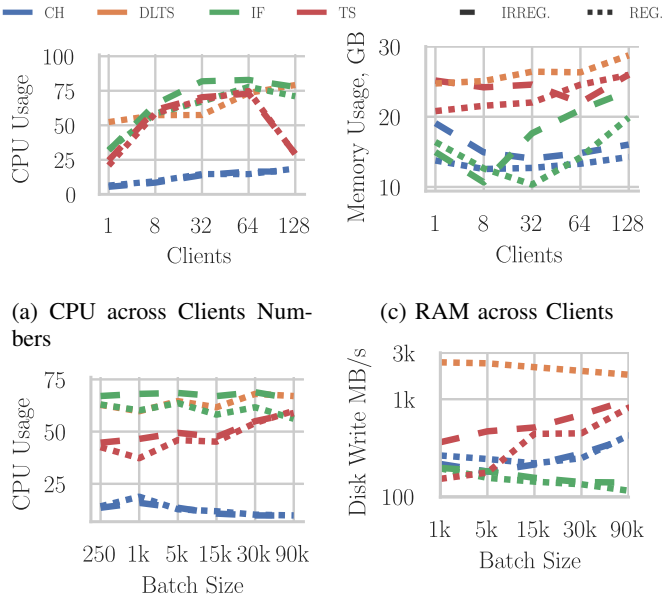
Firstly, we note an extreme low transmission speed for Timescale on single-value ingestion, which improves strongly until outperforming others on high-value-cardinal ingestion. Secondly, we generally observe rising performance with rising cardinality, stronger on the first six added values, and less between the 6th and the 24th added value. This could be due to diminished write amplifications, as the values are stored next to each other on the disk and fewer blocks must be rewritten, as shown in Figure 7(a). This is true for engines, which store values that are under one key, together. For ClickHouse, that would mean that granules, which are mapped to the same primary keys, are stored next to each other. Interestingly, Influx scales until six values, but then stops: The CPU bottleneck observed in Figure 7(b) prevents faster decompression for more than six values at a time. Instead, for DLTS we observe the contrary: With increasing value-cardinalities the ingestion rate sinks. DLTS is a vector-based system which is designed for single-value data, as the vector files comprehend one single dimension. Therefore, multiple values are written to distinct vectors and saved in distinct blocks, as one vector respects one time series and can easily take several blocks.

4) Ingestion Performance Across Various Length Of Time Series Per Batch: In Figure 4(a) we see the system's performance for various lengths of the time series inside a batch, under 128 clients. E.g., a batch can contain 60 measurements for one sensor value or, e.g., 300 measurements. If the frequency is 1 Hz, we would have one or, respectively, five minutes of data inside one batch. High amounts of measurements per batch, like several thousands, are interesting for either high-frequency measurements, like millisecond-scale measurements, or for low-bandwidth connections, which require better compression and larger buffers. While not of interest for the KATRIN use case, other scientific domains or industrial applications implement such kind of measurements.

We observe that most engines slightly improve performance, when packing multiple measurements of the same key in one batch. However, the observations vary strongly if we consider only one double float value per measurement, or 24. **For single-value-data points**, all engines but Timescale show a slight increase in performance. ClickHouse's benefit of longer TS per batch stops with 2k measurements, after which performance decreases slightly. DataLayerTS instead, once reached 600 measurements, outperforms other and continues to scale exponentially, until even reaching a higher magnitude of performance for extremely long time series per batches (which means automatically also high batch size). **Considering 24 values saved in each single data point**, the performances change significantly – for most systems, it is much higher: ClickHouse outperforms others, maintaining the characteristics observed before, but with generally higher ingestion rates. Influx instead decreases performance with increasing time series lengths, until it regains its performance when using very high lengths. TimescaleDB has very low performance until it packs more than a thousand measurements together, scaling rapidly and outperforming others just on extremely long time series. DataLayerTS, as shown in Figure 4(b), is much slower when saving multiple values per key, showing a similar performance curve to Timescale.

D. System Metrics - Ingestion

Why do some architectures with their implementations perform better than others? Apart from analyzing performance only, we analyze which system is outstandingly efficient, coping with far fewer resources than others. Are the system resources fully used? Where are the architecture-specific bottlenecks that limit certain techniques, but not others?



(a) CPU across Clients Numbers

(c) RAM across Clients

(b) CPU across Batch Sizes

(d) Disk write bandwidth in average.

Fig. 5: CPU and Memory consumption for equidistant and irregular workloads.

As we see in the measurements shown in Figure 5(a), 5(b) and 5(c), InfluxDB, Timescale and DataLayerTS are constantly high on CPU consumption, whereas ClickHouse is resource-efficient — this is true also considering Memory, with the difference that at least for low client numbers, Influx has a low

memory consumption. However, considering its competitive performance, ClickHouse is remarkably resource-efficient. We note that Influx's TSM-Tree represents a Tier-based LSM approach, while ClickHouse's Sparse Index MergeTree represents a Level-based approach [10]. As we see, Influx spends many CPU cycles – presumably to sort the values in order to write the TSM-files.

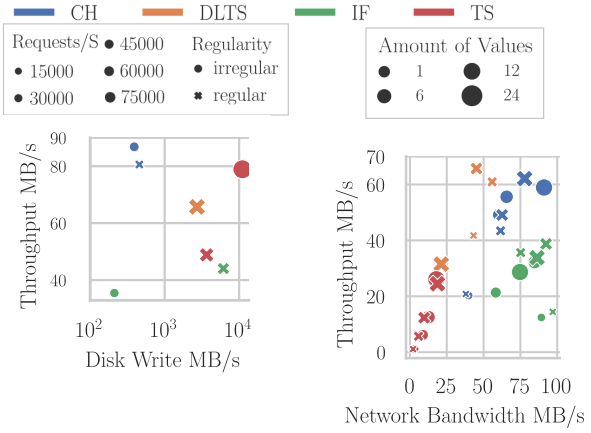
In Figure 5(d) we observe the saturation of the average disk input bandwidth, measured in bytes per second. Hardware benchmarks showed a max. disk speed of 1.2 GB/s. While DLTS consumes more than 1 GB per second, probably reaching hardware-imposed limits, other systems consume less bandwidth. Also, Timescale has high usage, while ClickHouse and especially Influx show very little bandwidth usage on average.

Figure 6(a): To fully understand how efficient these systems use disk I/O — the most critical limit in scalable, high-available systems — we analyze the disk input bandwidth usage, while considering the reached data transmission bandwidth. However, we note that Figure 6(a) does **not** show, e.g., DLTS under its best performance for extremely high batch sizes, but shows all values of all systems for the batch size of 15k. Similarly, just workloads from 128 parallel clients are of interest in the following section.

1) Metrics under 15k batch size and 128 Clients: Currently, KATRIN operates ca 10k sensors, each emitting one to 10 values a second. While not being a real-time operating slow control system, updates for the monitoring tools need to come with low latency, therefore, very big batch sizes would demand more buffering. Therefore, 15k is a realistic batch size for future KATRIN systems, which will operate more data than the current one. Our research group's aim is to store and manage data as directly from the sensors as possible, which makes it necessary for systems to manage many parallel clients. As the given benchmark hardware had 64 cores, we tested with 128 clients. Disk write amplification is a severe issue, as most high-availability data back-ends are based on distributed file systems, which have a hard-to-extend limit of IOPS (due to latencies introduced with distributed locking systems) and bandwidth.

We observe that for batch sizes of 15k, ClickHouse has the lowest disk write bandwidth usage while having the highest data transmission bandwidth; therefore, it operates most efficiently. For a similar data transmission rate, TimescaleDB needs a much higher disk write bandwidth, more than any other system. TimescaleDB's irregular data workloads are remarkably less efficient. Influx presents an interesting divergence: Regular transmission is faster and consumes more disk bandwidth than irregular transmission, which instead is the slowest measurement on this graph. We will try to understand the reason in the following analysis.

For high batch sizes, performance increases, but TimescaleDB hits the disk I/O bottleneck. Even more, and for the same reason, as shown in Figure 5(d) DataLayerTS will also bottleneck in disk writes because it has to rewrite entire vectors for each update, but due to its vector compression,



(a) Disk write bandwidth usage with data throughput, and disk write request count.

(b) Network and data throughput, across value-cardinalities.

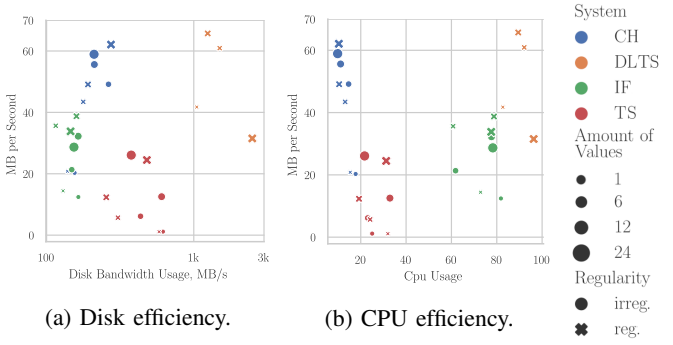
Fig. 6: Resource Consumption, compared with ingestion transmission speed, under 128 clients and 15k-sized batches.

it can use even higher batch sizes to work around this issue and outperform others in such special configurations.

In Figure 6(a) we see, displayed with the various sizes of the markers, how fragmented the system's disk access is. Timescale-irregular hits I/O bottlenecks, having a fragmented access pattern, which will be a problem in distributed file systems, due to their higher latencies. In contrast, ClickHouse is remarkably efficient in using disk write bandwidth, split only in few requests. Interesting is the big divergence of Influx-regular and irregular. Both are slower than the competitors, but only irregular data ingestion is using a very high disk bandwidth (Influx has a delta-based timestamp compression technique, which contributes to this divergence, as redundant deltas are more effective to compress).

In Figure 7(b) we see the relationship between CPU consumption and throughput. This has the scope to assess a system's capability to efficiently (not just effectively) process data ingestion. The measurements show 15k batch size under 128 clients.

After disk I/O-awareness, efficient data *processing* is fundamental for scalable systems. How complex is the creation of a database record, its maintenance, and update? We note that no system has (remarkably) more difficulties for regular than for irregular data. Influx and DataLayerTS have the highest CPU consumptions, followed closely by Timescale. Unfortunately, Influx provides the least data throughput – therefore, as analyzed in Figure 5(d), we assume that Influx hits quite early a CPU bottleneck, as the big, time-sorted SSTables seem to be quite difficult to update, because it means resorting a large file. This engine shows to be designed for low write amplification, with the trade-off of requiring more CPU power. This means that whenever disk I/O bandwidth is considerably lower, Influx would have a better stand against competitors than under the current hardware setup.



(a) Disk efficiency. (b) CPU efficiency.

Fig. 7: Resource consumption, compared with transmission speed, across value-cardinalities, under 128 clients and 15k-sized batches.

Most interesting is the low computing (in addition to low disk I/O) requirements of ClickHouse, which nevertheless maintains competitive throughput. As shown in Figure 6(a), ClickHouse's SparseTree engine, which stores each ~8k values together in a granule (array) and maps them to an index key, is under this workload configuration the most efficient compromise between easily writable and updatable files, and low compression and index necessities.

2) System Metrics Under Various Data Cardinalities: Does grouping of values under a single primary key help systems to be more efficient? To what extent?

In Figure 7(a), we observe that Timescale benefits most from higher value-cardinality, as while increasing performance it reduces written bytes per second — although for regular TS, medium-size value-cardinality is more efficient than the highest one. For ClickHouse and Influx, we observe an increase of performance with increasing value-cardinality, while maintaining stable disk bandwidth usage. DataLayerTS instead suffers from this kind of grouping: Data transmission speeds reduce severely and disk bandwidth increases with large group sizes. However, for 15k batches, even DataLayerTS profits from small group sizes — an interesting finding, after analysis of Figure 4(b). Although DLTS writes just one dimension per file, the system has (for low- and medium-sized batches) the possibility to buffer more data before check-pointing, when multiple values per data point are incoming. Here, in contrast to very high batch sizes, the transmitted one value per data point, under the standard checkpointer frequency, has not yet pushed the process to the I/O bottleneck.

In Figure 7(b) we see how various value-cardinalities affect computation efficiency. We observe that ClickHouse's and Timescale's biggest marker (24 v/DP)[18] is shown to be more performative while consuming similar resources than their smallest markers (1 v/DP). ClickHouse already benefits a lot from small groups (increasing from 1 to 6), while Timescale benefits most from large group sizes (increasing from 12 to 24) — and even saves some computing costs. Influx does not show any clear pattern; however, it prefers middle group sizes or regular TS.

Figure 6(b) shows the relationship between data sent in the network and the actual data throughput for 15k batches under 128 clients. We note that for all system's clients, transmission compression is turned off if this feature was available.
We can see here that most systems have similar bandwidth usage, just DataLayerTS consumes far less network bandwidth. The specialized vector-based batch, which does not contain single timestamps, is particularly useful for applications with low network bandwidth. Also, TimescaleDB consumes less network – one reason is the NPGSQL client, which has a particular batch ingestion serialization. Apart, Influx diverges strongly with having regular time series' network consumption far above the others (in both network usage and throughput), including its irregular data transmissions (as regular data is easily convertible to a sorted list, this memory structure shows to be more compatible). For ClickHouse instead, irregular TS transmits more and faster. For DataLayerTS, Timescale and mostly Influx, higher amounts of values saved in a single data point (higher value-cardinalities) are more efficient in network bandwidth usage, while for ClickHouse this is not the case.

E. Query Performance

1) Query Performance Under Regular And Irregular Time Series: The systems show just negligible differences in processing regular vs. irregular time-series data retrieval.

2) Query Performance Under Various Data Cardinalities: Disclaimer: Currently, aggregations across multiple values are not supported in the current version of SciTS. Aggregations mean here to aggregate along one column in a table, where one or multiple other values are saved, and see how a system is performative in aggregating a single value across time, in either a table with very wide data points (many values per data point) or more narrow ones (few or a single one). For a future version of SciTS, two-dimensional aggregations (across specific values in multi-value data points, across time) would be interesting to implement.

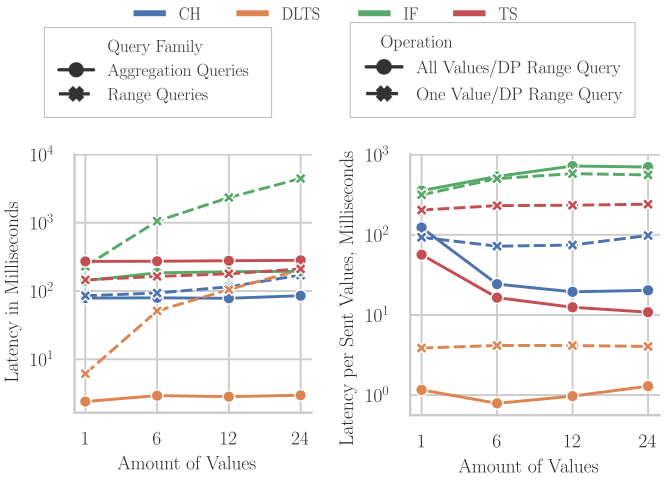
In Figure 8(a) we can see the query latency for both aggregation queries and range queries, operating in tables that contain various amounts of values per data point. This assesses the system's ability to extract data from single- or jointly stored values.

Regarding singly stored values, we see that Timescale has the highest latency values on aggregation query types, followed by Influx. DataLayerTS has astonishingly low response times for aggregation queries, outperforming others by a magnitude.

For 1-value range queries, Influx has the highest latencies. It uses more CPU and I/O resources for the file lookup, as higher-level TSM-files can get very large.

We observe that no system suffers in doing aggregations across tables with various value-cardinalities. Therefore, for every aggregation-heavy use case, there is no negative effect from grouping values together as a single data point in terms of retrieving them for calculations. For range queries, Timescale

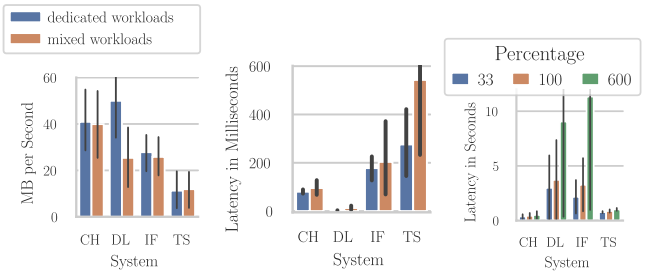
[18] v/DP = value(s) per data point.



(a) Aggregation and range query latencies.

(b) 1-value vs. all-values query data transmission speeds.

Fig. 8: Query performance in tables with increasing value-cardinalities.



(a) Ingestion throughput, 15k-sized batches.

(b) Aggregation query latency.

(c) Range query latency: output-relative-to-input data point quantities.

Fig. 9: Performances under mixed workloads, under 128 clients.

and ClickHouse show quite constant latencies, regardless of whether 1,6,12 or 24 values are retrieved at once, per timestamp of a given range (here, 1h). Therefore, these systems benefit from grouping. For DataLayerTS and Influx, instead, latencies increase with increasing group size. To understand whether these queries suffer from grouping, we look at Figure 8(b)

In Figure 8(b) we focus on range queries and see whether it is more performative to retrieve multiple values stored separately or jointly (under various group sizes).

In Figure 8(b) we observe that Timescale and ClickHouse benefit from querying ranges of jointly stored grouped values, rather than an equivalent amount of single-stored values. While Timescale stores these values in the same pages and accelerates retrieval, ClickHouse seems to be able to index them easier, or stores these related granules in subsequent blocks. Also, DataLayerTS shows a very small advantage in storing specifically small groups. For DLTS, being an engine that stores just one value-per-data point in an array, this is an interesting behavior – it comes from the database system optimizing better the more jointly arriving requests, not yet having reached the I/O bottleneck, a pattern already observed in Figure 7(a). Influx, instead, suffers from any kind of poly-value-cardinality: For every kind of group size, the latencies per retrieved value increase. In TSM files, these values are indexed distant from each other due to a group-agnostic sort algorithm, a factor which could be resolved or eased with correct tuning or using a specialized algorithm. Unfortunately, what Influx calls “spatial” partitioning has not been configured and doing so could have a positive influence on these workloads.

F. Mixed Workloads

In Figure 9(a) and (b), we observe how ingestion throughput and aggregation query latencies are influenced by ongoing read and, respectively, write operations. We note that the systems mostly cope well, however, DataLayerTS’ ingestion and Timescale’s aggregation performance decrease strongly with ongoing mixed workloads.

In Figure 9(c), we see the influence of the ongoing ingestion of various magnitudes on the range query latency. 33 percent means that for ingestions with, e.g., 1000 data points per batch, the ongoing limited query yields 333 data points.

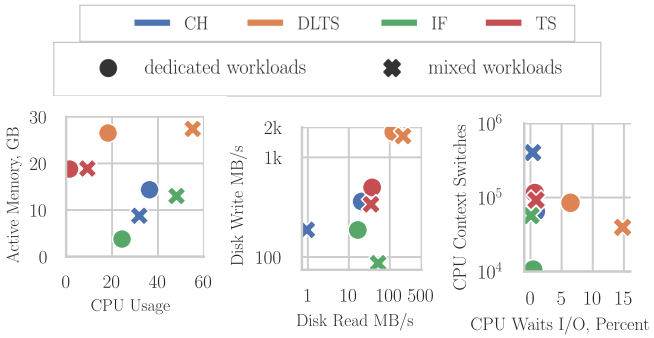
We see that most systems cope well with contemporary mixed workloads of various magnitudes; Timescale and DataLayerTS instead decrease strongly retrieval performance, especially on 600%, which means, as soon as the system retrieves 6 times the quantity of the contemporarily ingested data points, the retrieval slows down extremely. These observations are strongly related to the I/O bottlenecks observed in previous graphs.

For the other systems, ingestion or query speeds do not decrease with ongoing query operations, however large the queried ranges are.

1) System Metrics Under Mixed Workloads: In Figure 10(a), we observe that DataLayerTS uses much more CPU resources and Influx uses more CPU and Memory, when operating under mixed workloads. Interestingly, ClickHouse reduces memory usage.

In Figure 10(b), we observe that ClickHouse’s disk read bandwidth usage decreases under mixed workloads. Influx, instead, decreases in write bandwidth usage.

In Figure 10(c) we observe, that DataLayerTS spends more time waiting for I/O than other systems, especially when under mixed workloads – up to 15 percent. Another confirmation of the previously described disk bottleneck. ClickHouse shows many CPU context switches, when under mixed workloads. As we did not observe drastic increase in resource usage (actually, we observed a decrease in memory usage), this



(a) Memory and CPU, under 128 clients.

(b) Disk input and output bandwidth

(c) CPU idles and context switches

Fig. 10: Resource consumption under mixed workloads

Architecture	Timescale Row-B-Tree A.	ClickHouse Column-Sparse I. A.	Influx Key-Value TSM A.	DLTS Vector-TIV A.
Bottlenecks	Disk	not identified	CPU	Disk (if small batches)
Equidist. TS Ingestion	–	(✓)	✓	✓+
Value-[19] Cardinality	I: ✓ | Q: ✓ best: 24 | 24	I: ✓ | Q: ✓ best: 24 | 12	I:(✓) | Q: – best: 6 | 1	I:(✓) | Q:(✓) best: 12 | 6
Mixed Workloads	slower output	✓	✓	slower input
Storage	4th	2nd	3rd	1st
Perform.[20]	27 MB/S	62 MB/S	39 MB/S	65 MB/S

✓ = benefits with given workload characteristic, (✓) = benefits to a small extent or with drawbacks in efficiency, ✓+ = benefits strongly, – = suffers from given workload characteristic, Q.: Query | I.: Ingestion

TABLE III: Final, Summarized Evaluation

system copes particular well under mixed workloads and shows to be optimized for such cases.

VII. Conclusion

Monitoring systems of scientific experiments and any high-bandwidth IOT infrastructures, especially when they need to be highly available, profit from an efficient *and* performative implementation of a time series-specialized database system. Therefore, this research identified four common characteristics of data flows and workloads in which no prior study has been conducted. An existing benchmark tool has been extended to investigate these characteristics, which are the following. Equidistant time series, multivariate time series (means, multiple values stored under the same key), online/mixed workloads, and time series batches, having various length of time series. Furthermore, various batch sizes and amounts of parallel clients have been considered. Using the new SciTS-v2 benchmark, we investigate these six dimensions in benchmarking time series databases based on a quantitative and qualitative analysis of performance and efficiency profiles and bottlenecks — presented below, in Table III. This helps system architects to assess the most suitable architecture for their application, especially when their requirements might change in the future and they have a long lifetime. In addition, based on our studies, we propose optimal workload configurations for each of the four systems studied, based on our setup and requirements, shown in Table III.

The most promising candidate for applications with pure equidistant data and which can configure workloads with high batch sizes (from buffered data or high-frequency readings) and where data is coming from many parallel clients, is DataLayerTS. As soon as the clients send more than 300 timestamps per batch, this system outperforms any other system. In addition, it reaches or outperforms the other systems in all query workloads and consumes minimal storage space. For most other applications, which have more real-time requirements, need to pay more attention on computing resource consumption, or need more flexibility in the type of involved data, ClickHouse is the most suitable solution, as it is computationally most efficient, while being performative in all types of workload configurations.

To summarize, monitoring systems remarkably improve overall efficiency of the current data model, as well as simplifying its setup, by building on a column-based OLAP system, like e.g., ClickHouse, as storage engine and therefore allowing the system to cope with future, higher workloads. It's column-design, made for online analysis workloads, proves to suit well to current requirements and costs far less CPU/memory and maintenance. ClickHouse operated far below any resource limits – this lets us assume that also in peak workload situations, the application will not lose data: Reliability is a very crucial constraint of slow-control systems.

As the current database system, Ms-SQL, is a row-based architecture using B-Tree indices, we assessed that some improvements in performance can be made without changing the storage architecture, just by re-configuring the multi-value-cardinal data model to the optimal parameters, illustrated in Section III— although the CPU and I/O bottlenecks, which come with this architecture, not permitting the system to scale a lot. In addition, we assessed that by fully implementing a resolution-based, regular time series data model, the pipeline reaches further magnitudes of I/O performance and storage efficiency, when combined with a vector-based store, like e.g., DataLayerTS. It's array-design minimizes the best temporal index overheads, while still fitting to data analysis requirements of our use case.

[19] "best" means: highest throughput under 15k batch, 128 clients (our use case), first value: for ingestion WL, second: Query WL

[20] 15k batch, 128 clients

REFERENCES

[1] J. Mostafa, S. Wehbi, S. Chilingaryan, and A. Kopmann, "Scits: A benchmark for time-series databases in scientific experiments and industrial internet of things," in *Proceedings of the 34th International Conference on Scientific and Statistical Database Management*, ser. SSDBM '22. New York, NY, USA: Association for Computing Machinery, 2022. [Online]. Available: https://doi.org/10.1145/3538712.3538723

[2] T. K. collaboration, M. Aker *et al.*, "The design, construction, and commissioning of the katrin experiment," *Journal of Instrumentation*, vol. 16, no. 08, p. T08015, Aug. 2021. [Online]. Available: http://dx.doi.org/10.1088/1748-0221/16/08/T08015

[3] W. Eppler, A. Beglarian, S. Chilingaryan, S. Kelly, V. Hartmann, and H. Gemmeke, "New control system aspects for physical experiments," *Nuclear Science, IEEE Transactions on*, vol. 51, pp. 482 – 488, 07 2004.

[4] N. K. J. A. E. A. et al., "Status of j-parc operation data archiving using hadoop and hbase," *10. annual meeting of Particle Accelerator Society of Japan; Nagoya, Aichi (Japan)*, 2013.

[5] W. Shen, Y. Chen, H. Zheng, J. Wang, G. Shen, and G. Huang, "A new epics time-series data archiver using hbase," *Radiation Detection Technology and Methods*, vol. 5, no. 4, pp. 550–557, 2021.

[6] I. Finch, B. Aljamal, K. Baker, R. Brodie, J.-L. Fernández-Hernando, G. Howells, A. Kurup, M. Leputa, S. Medley, and A. Saoulis, "Vsystem to EPICS Control System Transition at the ISIS Accelerators," in *Proc. 13th International Particle Accelerator Conference (IPAC'22)*, ser. International Particle Accelerator Conference, no. 13. JACoW Publishing, Geneva, Switzerland, 07 2022, pp. 1156–1158. [Online]. Available: https://jacow.org/ipac2022/papers/tupopt063.pdf

[7] Y. Qiao, G. Lei, and Z. Zhao, "The Study of Accelerator Data Archiving and Retrieving Software," in *8th International Particle Accelerator Conference*, 5 2017.

[8] Y. Hao, X. Qin, Y. Chen, Y. Li, X. Sun, Y. Tao, X. Zhang, and X. Du, "Ts-benchmark: A benchmark for time series databases," in *2021 IEEE 37th International Conference on Data Engineering (ICDE)*, 2021, pp. 588–599.

[9] P. Gupta, M. Carey, S. Mehrotra, and R. Yus, "Smartbench: a benchmark for data management in smart spaces," *Proceedings of the VLDB Endowment*, vol. 13, pp. 1807–1820, 08 2020.

[10] C. Luo and M. J. Carey, "Lsm-based storage techniques: A survey," *CoRR*, vol. abs/1812.07527, 2018. [Online]. Available: http://arxiv.org/abs/1812.07527

[11] A. Persaud, M. J. Regis, M. W. Stettler, and V. K. Vytla, "Control infrastructure for a pulsed ion accelerator," *IEEE Transactions on Nuclear Science*, vol. 63, no. 5, pp. 2677–2681, 2016.

[12] R. Liu and J. Yuan, "Benchmark time series database with iotdb-benchmark for iot scenarios," *CoRR*, vol. abs/1901.08304, 2019. [Online]. Available: http://arxiv.org/abs/1901.08304

[13] A. Khelifati, M. Khayati, A. Dignös, D. Difallah, and P. Cudre-Mauroux, "Tsm-bench: Benchmarking time series database systems for monitoring applications," *Proceedings of the VLDB Endowment*, vol. 16, pp. 3363–3376, 08 2023.

[14] C. Inc., "What is clickhouse? — clickhouse docs." https://clickhouse.com/docs/en/intro/, online. Accessed: 8.7.2024.

[15] S. Papadopoulos, K. Datta, S. Madden, and T. Mattson, "The tiledb array data storage manager," *Proc. VLDB Endow.*, vol. 10, no. 4, p. 349–360, nov 2016. [Online]. Available: https://doi.org/10.14778/3025111.3025117

[16] M. Poess, R. Nambiar, K. Kulkarni, C. Narasimhadevara, T. Rabl, and H.-A. Jacobsen, "Analysis of tpcx-iot: The first industry standard benchmark for iot gateway systems," in *2018 IEEE 34th International Conference on Data Engineering (ICDE)*, 2018, pp. 1519–1530.

[17] T. Toye, "Towards a representative benchmark for time series databases," Master's thesis, Ghent University, 2019.

[18] "Influx comparison: Benchmarking influxdb against other databases and time series solutions," https://github.com/influxdata/influxdb-comparisons, online. Accessed: 8.7.2024.

[19] T. Inc., "Time series benchmark suite," https://github.com/timescale/tsbs, online. Accessed: 8.7.2024.

[20] C. Inc., "Clickbench: a benchmark for analytical databases," https://github.com/ClickHouse/ClickBench, online. Accessed: 8.7.2024.

[21] H. Chen, G. Liu, D. Zhang, T. Qin, and X. Sun, "A study of performance comparison of databases for half data archiving system," *Journal of Instrumentation*, vol. 18, p. P12015, 12 2023.

[22] N. Hennion, "Glances," https://github.com/nicolargo/glances, online. Accessed: 8.7.2024.